

Automated Planning for Self-Adaptive Systems

Richard Gil

INESC-ID, Instituto Superior Técnico, Universidade de Lisboa
richard.martinez@tecnico.ulisboa.pt

Abstract—Self-adaptation has been proposed as a viable solution to alleviate the management burden that is induced by the dynamic nature and increasing complexity of computer systems. In this context, architectural-based self-adaptation has emerged as one of the most promising approaches to automatically manage such systems, resorting to a control loop that includes monitoring, analyzing, planning, and executing adequate actions. This work addresses the challenges of *adaptation planning* -the decision-making process for selecting an appropriate course of action- with a focus on the problem of provisioning automated mechanisms for assembling adaptation plans, as a means to enhance adaptive capabilities under uncertainty. To this purpose, adaptations are modeled in a hierarchical manner, defining primitive actions, guarded actions, and deliberate plans, which may guide the system towards a desired state.

I. INTRODUCTION

Complexity in computer systems has been increasing dramatically in the last decades, to the point of becoming a limiting factor for their further development. Companies employ large-scale distributed systems, which are ran manually by a growing-in-size staff, in a time-consuming, expensive, and error prone manner. Thus, industry demands changes in the way systems are engineered, so that management won't become a technical and economical impediment for their use.

To avoid the management bottleneck, complex tasks such as: adapting the system to user requirements, tuning parameters to optimize system's performance or to accommodate to resource availability, enabling strategies to react to systems failures; can no longer rely solely on human effort, thus demanding some self-adaptive capabilities in the system. Ideally, an automated solution that leverages (human) domain expertise, provides a global perspective on the system, adjust the system opportunely and effectively, and can be engineered without major cost, is desired.

Autonomic Computing aims at reducing the barrier that complexity poses to further development of such systems by integrating intelligence into the management process[1]. By closing the operational loop via an external control mechanism[2], it is possible to monitor the performance of the system, analyze its behavior, and, when necessary, plan and execute proper adaptation actions to guide the system towards business-defined goals. The MAPE-K model identifies four (K)nowledge-based enabled tasks in the loop: (M)onitoring, (A)nalyzing, (P)lanning, and (E)xecuting[1]. How to engineer such mechanisms remains an open question till today.

Previous work has put in evidence that current systems are still unable to fully exploit the power of intelligent planning in order to achieve optimal system operation[3]. While corrective

actions can be defined at a low level of control, the question on how to automatically elaborate high-level deliberate plans, able to consider environmental deviations and to guide a system under uncertainty, is still unanswered. This work addresses the challenges of automatic generation of operationalized plans that take uncertainty into account. In detail, my work is directed to exploring how intelligence can be integrated into planning, so as to generate adaptive plans to guide the system to an optimal state, which satisfies business-defined goals.

The rest of the paper is structured as follows. The next section briefly discusses related work. Section III presents my approach for the automatic generation of deliberate plans. A case of study is presented in Section IV. Finally, Section V discusses the current status and the research plan.

II. RELATED WORK

Among several approaches to engineering self-adaptive systems, the architectural model has been accepted in the community as a mean to support reasoning about the system, mainly due to its expressiveness and generality, as well as a widely explored solution, given its potential for re-usability and scalability[4][5][6]. From such perspective, the managed system can be described as a set of components to which metrics, actions, and rules can be associated, and some constraints can be posed. Multiple works[7][8][9] have illustrated the benefits of the approach for engineering control loops, via monitoring metrics, analyzing them according to pre-defined rules, and executing actions to correct system's operation.

When compared to other frameworks, Rainbow[10] is the most complete, enabling self-configuration, self-optimization, and self-healing. *Architectural styles* are used to enforce the reuse of style-specific monitoring techniques (which data is extracted and mapped), repair rules (how to adapt to constraint violations), and adaptation operators (primitive actions to repair the system). Also, the planning task is decoupled from the actual sensors and effectors, making Rainbow a candidate to experiment with different decision-making techniques.

Recently, Cheng et al.[11] have proposed Stitch, a language for architecture-based adaptations that succeeds in incorporating expert's knowledge in the decision-making process, by enabling the concepts of *operator*, *tactic*, and *strategy* hierarchically. This approach relies on predictions on the impact of adaptations to perform a utility-based selection of the best-matching strategies at run-time. Stitch, however, remains limited to the expert's knowledge about the system and its environment to define a limited pool of strategies. This could compromise the autonomic nature of the system when in

presence of unforeseen behavior or when none of the strategies in the pool is able to successfully conduct the system to the final desired state.

Rosa et al.[12] promote a hierarchical decision-making task, by defining *adaptations* and *adaptation sets* (intended as the set of all adaptation that may be executed concurrently). The authors propose a component-based approach which: (a) extracts adaptation rules offline, by selecting all valid adaptation sets which may have a positive impact on the system when a particular goal-constraint is violated, and (b) selects at runtime the adaptation set best matching a (ranked) set of goals that the system should meet. While this system benefits from an automated rule generation procedure, able to foresee all possible adaptations at any valid state, it lacks of any planning capability, limiting adaptation to a greedy execution of rules prone to reach local optima only.

For both of these proposals[11][12], an automatic mechanism for assembling deliberate plans, which is able to anticipate all possible adaptation paths, to grade them according to some customized criteria, and to select the best path, is desirable. Such plans must be elastic enough so as to consider the environmental effects over the system, and general enough to be used under general applicability conditions. Additionally, utility-based plan selection may be used to swap to a less “ambitious” plan whenever a best-match plan breaks down.

Some authors have already explored the planning problem in similar scenarios. In [13], the authors propose to generate reactive plans using a planning-model-checking tool. In this approach there is no correspondence between plan states and component-specific actions, i.e. planning is done under the assumption that planned actions “can” indeed be implemented by some component of the system, and a further non-trivial mapping step is required to derive component configurations. Also, planning remains limited to a unique technique and tool.

In my research, I also intend to explore planning, by using several techniques and tools (planners), aiming at avoiding the drawbacks of current proposals by integrating results from related fields to automate the generation of adaptive plans, departing from component-specific expert-defined actions. To the purpose of my proposal I take advantage of work from:

a) *Automated planning*: From the artificial intelligence community, automated planning is rich in models, languages, and planners for addressing planning in dynamic systems. PDDL[14], the lingua franca of planning, is of particular interest for specifying domain and problem representations, and has been widely used for implementing planners using the most varied techniques. Some related work attempts to encode software architectures problems into PDDL[15][16][17]. In [18] a software architecture evolution problem is mapped to PDDL and the authors demonstrate how automated planners can effectively support automatic generation of evolution paths. However, questions regarding modeling uncertainty, modeling bigger adaptation constructs, and generating multiple paths, remain unanswered.

b) *Planning under uncertainty*: An important issue when executing adaptation actions in dynamic systems is

uncertainty[19]. To cope with the difficulty of predicting the system next state after an action is executed, most authors opt for modeling the system by defining probabilistic state transitions. In this line are some relevant proposals: in [20] an automaton is augmented with probabilities and the executor chooses among alternative paths; in [21] a probabilistic model checker is used for the analysis of quantitative properties of the system; in [22] control in a DTMC model is combined to changes that may be expressed as probability transitions; finally, in [11] probabilities are associated to the branches of a tree-like conditional expert-defined plan, called strategy. Furthermore, some authors tackle the problem of plan breakdown due to (unexpected) environmental effects and propose solutions for: (a) re-planning, e.g. via utility-based[11] or goal rank match[12] plan re-selection; or for (b) planning to guarantee graceful degradation, by combining realistic and idealized models[23].

c) *Learning from the environment*: The expert’s knowledge may not always suffice to properly model the system and its environment. Such models can be incomplete or inaccurate and they are often be out of date. To tackle this limitations, some authors have proposed incorporating learning techniques to keeping models correct and up-to-date. Relevant work in this direction includes: a probabilistic rule learning approach to update models using feedback in the form of execution traces [24] and a framework for learning the impact of adaptation decisions on the system’s goals [25].

To the best of my knowledge, the proposal of integrating hierarchical adaptation policies (primitive, reactive, and deliberate) with automated planning tools for generating adaptation strategies is novel.

III. APPROACH

I adhere to an architectural approach for engineering self-adaptive systems. Similarly to [12], I describe the system in terms of component-specific metrics, adaptation actions, and goal policies. Reasoning from a hierarchical perspective[6], adaptation is defined at three control levels as: primitive actions (operators), reactive or guarded actions (tactics), and deliberate plans (strategies). As described in [11]:

- *Operator*: low-level component-specific command.
- *Tactic*: middle-level construct encapsulating an operator with its respective pre-conditions, post-conditions, and impact functions. It serves as a logical unit for specifying the cost and benefit of an adaptation action.
- *Strategy*: high-level tree-like deliberate plan, where each step is the conditional execution of some tactic. It consist of a set of paths and forking points to guide the system from an initial to a final state.

In such context, it is my purpose to describe all operators, pre-conditions, and post-conditions in a standard planning language, so as to enable deterministic automated planning tools for finding a sequence of tactics (adaptation paths) that can be further packed as deliberate plans (strategies), to which valid applicability conditions and real-time measured impact are associated. I proceed in three stages:

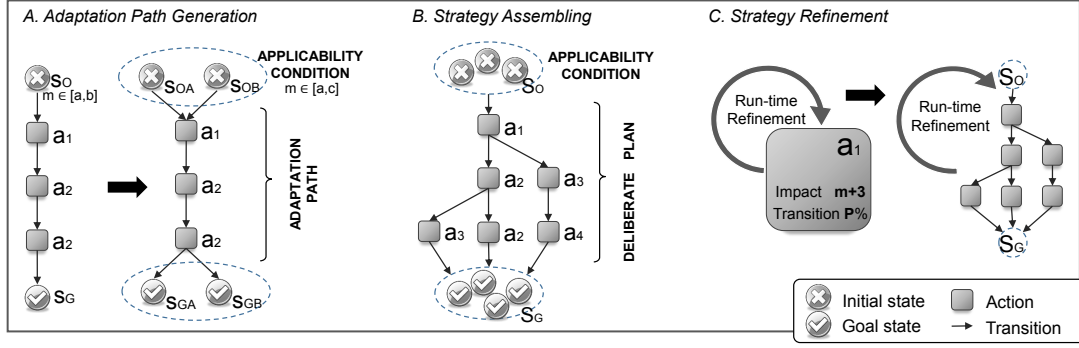


Fig. 1. Approach to strategy assembling and refinement

A. Adaptation Path Generation

Given a set of states S , a set of actions $A : S \rightarrow S$, an initial state $s_0 \in S$, and a set of goal states $S_G \subseteq S$, the *classical planning* problem is the task of finding a sequence of actions that, when applied to s_0 , yield one of the goal states. Thus, the planning problem can be framed as follows:

- S , the set of *states*, is defined to be the set of all legal system configurations, i.e. the combination of all possible values of the system metrics.
- A , the set of *actions*, is defined to be the set of expert-defined adaptation operators, with their respective pre-conditions, post-conditions, and impact functions. They represent *tactics* as defined previously.
- s_0 , the *initial state*, is defined to be any state of the system in which (at least) one of the metrics does not satisfy the system per-metric declared goals.
- S_G , the set of *goal states*, is defined to be a list of all system states that satisfy the per-metric declared goals.

With the problem framed this way, I feed several off-the-shelf planners with a standard-formatted PDDL specification, consisting of a description of the problem domain (component types, predicates, metrics, actions, and impact functions) and of the problem case (components, initial state, goal state, and a cost function). By running planners for several initial and goal states, under different parametrizations of the cost function, I am able to generate a set of candidate adaptation paths.

Given that multiple sequences may be similar, I further aggregate such fine-grained paths into coarser paths, which applicability correspond to a bigger group of initial states. To this aim, clustering mechanisms are executed for value segmentation, so as to identify the best range subdivision for each system metric. Doing so, selected adaptation paths are (fairly) the same under a wider range of values. The aim is to generate a pool of adaptation paths able to efficiently guide the system from a (general) initial condition to a desired final state. For the sake of generality, some optimal paths may be sacrificed and a sub-optimal, but still valid path, be selected.

B. Strategy Assembling

In order to incorporate uncertainty into the adaptation plans, I model system transitions as probabilistic steps, such that,

after the execution of an action, different potential next states can be predicted with certain probability. I model the impact of actions by assigning impact values and probabilities[26]. I rely on expert estimations as a first approximation.

Once next states are identified for each state of the system, planners are used to generate alternative adaptation paths for each of these next states. This results in a tree of candidate paths with associated probabilities. When integrated into a closed-loop adaptation framework, a selected strategy guide the execution of the plan, so that after each adaptation step the system evaluates which adaptation path (if exist) to follow.

C. Strategy Refinement

I propose to update generated strategies by first refining expert-estimated impact functions and probabilistic effects at run-time (using learning mechanisms) and then adjusting strategies accordingly. For this purpose, the system will dispose of collected data about the system and its environment.

I compute at run-time the difference between the system metric values before and after each action is executed. The differences stored in a log are analyzed by a function estimator, to define the math nature of the effects an action has over each system metric. A similar approach is proposed for refining transition probabilities, based on the frequency of occurrences.

My approach ensures that it's possible to automatically generate deliberate plans departing from expert-defined guarded actions, impact functions, and high-level goals. In contrast to [12] my planning approach is not limited to an eager execution of actions (which may lead to local optima only). Instead, the generated strategies are able to deliberately guide the system to a global optima. Furthermore, online refinement ensures that strategies are kept valid and up-to-date. It is worth noting that, even when I plan to explore offline strategy assembling, I believe this to be a preliminary step towards engineering such mechanisms to operate entirely at run-time.

IV. INITIAL CASE STUDY

As an initial case-study, to validate the approach I will use an online retail website that follows a multi-tier architecture[12]. Webpages are customized to the user type, while components in the back-end deal with different content

types (e.g. high/low quality image). The system's purpose is to satisfy a set of goals (e.g. to limit query load, to maximize content quality, to minimize response time) by performing actions (e.g. change image quality) that affect performance metrics (e.g. query load, image resolution, avg. response time). It is my expectation that following the proposed approach I am able to generate a pool of strategies that is effective at guiding the system towards an optimal operational state. The proposed system will be evaluated under CPU overload and query rate overload scenarios. Specifically, I expect to see improvements over results obtained in [12]. Also, scalability and reaction time will be assessed. More complex case-studies, using realistic distributed key-value stores, will be used in subsequent phases of my work.

V. EXPECTED CONTRIBUTIONS AND ROAD AHEAD

The expected contributions can be enumerated as follows:

- *Adaptation path generation (offline)*: Algorithms for generating candidate adaptation paths from any state toward a goal state (using off-the-shelf planners), able to select the optimal path and to aggregate similar adaptation paths under common applicability conditions.

- *Strategy assembling (offline)*: Algorithms for generating alternate paths after executing an adaptation step and for further encapsulating a tree of such paths into a strategy, with applicability conditions and estimated impact over the system.

- *Strategy refinement (online)*: Algorithms for collecting and analyzing impact functions and transition probabilities at run-time, to refine generated plans accordingly.

So far, a first PDDL specification of the use-case has been described and trials over a selected off-the-shelf planners have been done. In the next months, I plan to develop techniques for generating a first set of adaptation paths and for aggregating them in coarser adaptation paths. After that, I plan to elaborate techniques to assemble strategies following the tree-like model. Finally, assembled strategies will be run online for refining expert's estimations.

ACKNOWLEDGMENT

This work was supported in part by the European Commission EACEA Grant for Erasmus Mundus Joint Doctorate in Distributed Computing under FPA2012-0030 and in part by Fundação para a Ciência e Tecnologia (FCT) under references UID/CEC/50021/2013 and CMUP-EPB/TIC/0042/2013. I am working under joint supervision of Prof. Luís Rodrigues (IST), Prof. Antónia Lopes (FCUL), and Prof. Peter Van Roy (UCL).

REFERENCES

- [1] J. O. Kephart and D. M. Chess, "The vision of autonomic computing," *Computer*, vol. 36, no. 1, pp. 41–50, 2003.
- [2] Y. Brun, G. D. M. Serugendo, C. Gacek, H. Giese, H. Kienle, M. Litoiu, H. Müller, M. Pezzè, and M. Shaw, "Engineering self-adaptive systems through feedback loops," in *Software Engineering for Self-Adaptive Systems*. Springer, 2009, pp. 48–70.
- [3] M. Salehie and L. Tahvildari, "Self-adaptive software: Landscape and research challenges," *ACM TAAS*, vol. 4, no. 2, p. 14, 2009.
- [4] D. Weyns and T. Ahmad, "Claims and evidence for architecture-based self-adaptation: A systematic literature review," in *Lecture Notes in Computer Science*, vol. 7957, 2013, pp. 249–265.
- [5] S.-W. Cheng, D. Garlan, B. Schmerl, J. P. Sousa, B. Spitznagel, and P. Steenkiste, "Using architectural style as a basis for system self-repair," in *Software Architecture*. Springer, 2002, pp. 45–59.
- [6] J. Kramer and J. Magee, "Self-managed systems: an architectural challenge," in *Future of Software Engineering, 2007. FOSE'07*. IEEE, 2007, pp. 259–268.
- [7] P. Oreizy, D. Heimbigner, G. Johnson, M. M. Gorlick, R. N. Taylor, A. L. Wolf, N. Medvidovic, D. S. Rosenblum, and A. Quilici, "An architecture-based approach to self-adaptive software," *IEEE Intelligent Systems*, vol. 14, no. 3, pp. 54–62, 1999.
- [8] S. Dobson, S. Denazis, A. Fernández, D. Gaiti, E. Gelenbe, F. Massacci, P. Nixon, F. Saffre, N. Schmidt, and F. Zambonelli, "A survey of autonomic communications," *ACM TAAS*, vol. 1, no. 2, pp. 223–259, 2006.
- [9] S.-W. Cheng, *Rainbow: cost-effective software architecture-based self-adaptation*. ProQuest, 2008.
- [10] D. Garlan, S.-W. Cheng, A.-C. Huang, B. Schmerl, and P. Steenkiste, "Rainbow: Architecture-based self-adaptation with reusable infrastructure," *Computer*, vol. 37, no. 10, pp. 46–54, 2004.
- [11] S.-W. Cheng and D. Garlan, "Stitch: A language for architecture-based self-adaptation," *Journal of Systems and Software*, vol. 85, no. 12, pp. 2860–2875, 2012.
- [12] L. Rosa, L. Rodrigues, A. Lopes, M. A. Hiltunen, and R. D. Schlichting, "Self-management of adaptable component-based applications," *IEEE Trans. Software Eng.*, vol. 39, no. 3, pp. 403–421, 2013. [Online]. Available: <http://doi.ieeecomputersociety.org/10.1109/TSE.2012.29>
- [13] D. Sykes, W. Heaven, J. Magee, and J. Kramer, "Plan-directed architectural change for autonomous systems," in *Proceedings of the 2007 conference on Specification and verification of component-based systems: 6th Joint Meeting of the European Conference on Software Engineering and the ACM SIGSOFT FSE*. ACM, 2007, pp. 15–21.
- [14] D. McDermott, M. Ghallab, A. Howe, C. Knoblock, A. Ram, M. Veloso, D. Weld, and D. Wilkins, *PDDL-the planning domain definition language*, 1998.
- [15] C. Rasche and S. Ziegert, "Multilevel planning for self-optimizing mechatronic systems," in *5th ADAPTIVE*, 2013, pp. 8–13.
- [16] C. E. da Silva and R. de Lemos, "A framework for automatic generation of processes for self-adaptive software systems," *Informatica: An International Journal of Computing and Informatics*, vol. 35, no. 1, pp. 3–13, 2011.
- [17] J.-E. Méhus, T. Batista, and J. Buisson, "Acme vs pddl: support for dynamic reconfiguration of software architectures," *arXiv preprint arXiv:1206.0122*, 2012.
- [18] J. M. Barnes, A. Pandey, and D. Garlan, "Automated planning for software architecture evolution," in *ASE 2013, IEEE/ACM*. IEEE, 2013, pp. 213–223.
- [19] A. J. Ramirez, A. C. Jensen, and B. H. Cheng, "A taxonomy of uncertainty for dynamically adaptive systems," in *SEAMS, 34th ICSE*. IEEE, 2012, pp. 99–108.
- [20] C. Ghezzi, L. S. Pinto, P. Spoletini, and G. Tamburrelli, "Managing non-functional uncertainty via model-driven adaptivity," in *Proceedings of the 35th ICSE*. IEEE Press, 2013, pp. 33–42.
- [21] R. Calinescu and M. Kwiatkowska, "Using quantitative analysis to implement autonomic it systems," in *Proceedings of the 31st ICSE*. IEEE, 2009, pp. 100–110.
- [22] A. Filieri, C. Ghezzi, A. Leva, and M. Maggio, "Self-adaptive software meets control theory: A preliminary approach supporting reliability requirements," in *Proceedings of the 26th IEEE/ACM ASE*. IEEE Computer Society, 2011, pp. 283–292.
- [23] N. D'Ippolito, V. A. Braberman, J. Kramer, J. Magee, D. Sykes, and S. Uchitel, "Hope for the best, prepare for the worst: multi-tier control for adaptive systems," in *36th ICSE*, 2014, pp. 688–699.
- [24] D. Sykes, D. Corapi, J. Magee, J. Kramer, A. Russo, and K. Inoue, "Learning revised models for planning in adaptive systems," in *Proceedings of the 35th ICSE*. IEEE Press, 2013, pp. 63–71.
- [25] A. Elkhodary, N. Esfahani, and S. Malek, "Fusion: a framework for engineering self-tuning self-adaptive software systems," in *Proceedings of the 18th ACM SIGSOFT FSE*. ACM, 2010, pp. 7–16.
- [26] J. Cámara, A. Lopes, D. Garlan, and B. Schmerl, "Impact models for architecture-based self-adaptive systems," in *Proceedings of the 11th International Symposium on Formal Aspects of Component Software (FACS2014)*, ser. LNCS, vol. 7795, 2015, pp. 89–107.